



INSTITUTO SUPERIOR TÉCNICO

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

ORGANIZAÇÃO DE COMPUTADORES

LEIC

First Lab Assignment: System Modeling and Profiling

Version 1.2

2025/2026

1 Introduction

The goal of this assignment is twofold: (i) to determine the characteristics of a computer's caches, and (ii) to leverage this knowledge to optimize the performance of a given program. For this task, students will use a performance analysis tool that provides direct access to hardware performance counters available on most modern microprocessors. The tool employed is the standard Application Programming Interface (API), **PAPI** [1].

In the rest of this section, we briefly introduce PAPI and describe the targeted computer platform and development environment. In Section 3, we present the procedure for modeling the L1 and L2 caches of the target platform (Subsection 3.1), and provide a guide for analyzing the performance of a matrix-multiply code segment and optimizing it based on the characteristics of the L2 cache of the target architecture (Subsection 3.2).

1.1 Targeted Platform and Development Environment

IMPORTANT: This assignment must be performed on the computers of your lab classroom. **Consult your lab instructor if you have any doubts.** These computers share similar hardware characteristics, and any of them may be used as the target platform. Since this work is hardware-dependent, using a computer with different hardware could lead to unexpected results and thus invalidate your work. Therefore, always use the same lab environment. If you are an Alameda student, you can access the specific lab computer you want (see <https://welcome.rnl.tecnico.ulisboa.pt/#labs-access>).

To set up the development environment, you will need the PAPI library and a set of auxiliary program files. This material is provided in the package **lab1_kit.zip**, available on the course website. After downloading and extracting the package on one of the lab computers, you must build PAPI. To do this, navigate to the source code directory **papi-X.X.X/src** and compile by running:

```
./configure
```

```
Make
```

This process generates a set of helper tools in the **src/utlis/** directory and creates the PAPI library **papilib.a**. Among the tools, `papi_avalis` is especially useful for checking which PAPI events are supported on the target platform. The library will later be linked to the auxiliary programs described in the following sections.

2 First steps

To help determine the characteristics of the lab's computer's caches, the following exercises will help you estimate cache parameters from small C applications.

The first step to get acquainted with the procedure is to determine only the size of the cache using a small C application on a (known) machine, such as the code you have analyzed on lab exercise VI.3. This C code is a simplified version of the following programs in this assignment. Basically, it iterates over an array to determine the cache size.

To guarantee that you measure the time accurately, please use the source code available in the lab kit (file `spark.c`).

In order to perform the evaluation you should go to your lab to access the cache size by running the application there. You may want to repeat the evaluation of the elapsed time a few times to achieve statistical significance. You should table the relevant results for different cache sizes on the response sheet and make a conclusion regarding the cache size. You can calculate more measures before the output, examine the final part of the source code file.

1. What is the cache capacity of the computer you evaluated? Please justify.

To discover the other cache parameters, you're going to modify the C application, so that it generates different data access patterns. Please spend a few minutes analyzing the modifications to the source code.

```
for(size_t cache_size = CACHE_MIN; cache_size < CACHE_MAX; cache_size = 2*cache_size) {
    for(size_t stride = 1; stride <= cache_size/2; stride = 2*stride){
        limit = cache_size - stride + 1;
        for(ssize_t repeat = 0; repeat < N_REPETITIONS * stride; repeat++) {
            for(index = 0; index < limit; index += stride) {
                array[index] = array[index] + 1;
            }
        }
    }
}
```

The meaning of each variable is the following:

array[] an arbitrary large array that will be repeatedly accessed to measure the cache miss pattern;

cache_size value of the cache size under test; all cache sizes given by integer powers of 2, between `CACHE_MIN = 8kB` and `CACHE_MAX = 64kB` should be considered;

stride states how many entries are being skipped at each access; for example, if the stride is 4, entries 0, 4, 8, 12, ... in the array are being accessed, while entries 1, 2, 3, 5, 6, 7, 9, 10, 11, ... are skipped;

limit the largest address that will be accessed for the cache size and access pattern under test;

repeat denotes the number of times that each access pattern will be repeated in an array.

This code segment was executed on an arbitrary machine. By varying the stride parameter and testing across different array sizes (ranging from `ARRAY_MIN = 4kB` to `ARRAY_MAX = 4MB`), the results from Figure 1 were obtained. Using solely the **information from the graph** answer the following questions:

2. What is the cache capacity of the computer?

A análise da Figura 1 revela a existência de dois comportamentos distintos nos tempos de acesso, consoante o tamanho dos arrays.

No primeiro caso, observa-se um tempo de acesso estável entre 300 ns e 500 ns, característico de arrays de menor dimensão. Este comportamento indica que esses arrays conseguem ser totalmente armazenados na cache, garantindo acessos rápidos e consistentes.

Já no segundo caso, os tempos de acesso sobem para valores aproximadamente entre os 900 ns e os 1000 ns, o que evidencia a ocorrência frequente de *cache misses*. Isto acontece porque arrays maiores ultrapassam a capacidade da cache, forçando o sistema a recorrer com maior frequência à memória principal.

O ponto de transição entre os dois cenários surge entre arrays de 64 KiB e 128 KiB, onde há um aumento acentuado no tempo de leitura e escrita. Esse salto é um forte indício de que a cache possui 64 KiB de capacidade, dado que esse é o maior tamanho de array que ainda pode ser inteiramente acomodado nela sem penalizações significativas.

3. What is the size of each cache block?

Para identificar o tamanho do bloco de cache, é necessário observar o primeiro valor de *stride* em que os tempos de acesso do grupo mais lento se tornam constantes.

Esse comportamento ocorre porque, a partir do momento em que o *stride* é igual ou superior ao tamanho do bloco, cada acesso passa a referir-se a um bloco diferente da cache. Isso provoca uma taxa de *miss* próxima de 100%, resultando em tempos de acesso uniformes para todos os *strides* seguintes.

Na análise, essa estabilização acontece a partir de um *stride* de 16 Bytes, o que indica que o bloco de cache possui exatamente 16 Bytes.

4. What is the L1 cache miss penalty time?

Para calcular a penalização associada a uma falta na cache L1, basta comparar o tempo de acesso quando todos os dados cabem na cache com o tempo de acesso quando há *misses* e o sistema é obrigado a recorrer à memória principal. A diferença entre estes valores traduz o custo de um *cache miss*.

Quando o array é menor do que a capacidade da cache, a taxa de *miss* é praticamente nula e o tempo de acesso ronda os 375 ns. Já para arrays maiores, combinados com *strides* iguais ou superiores a 16 bytes, a taxa de *miss* aproxima-se dos 100%, resultando em tempos de acesso em torno de 1000 ns.

Assim, a penalização de uma falta na cache L1 corresponde a $1000 - 375 = 625$ ns (L2 access time - L1 access time).

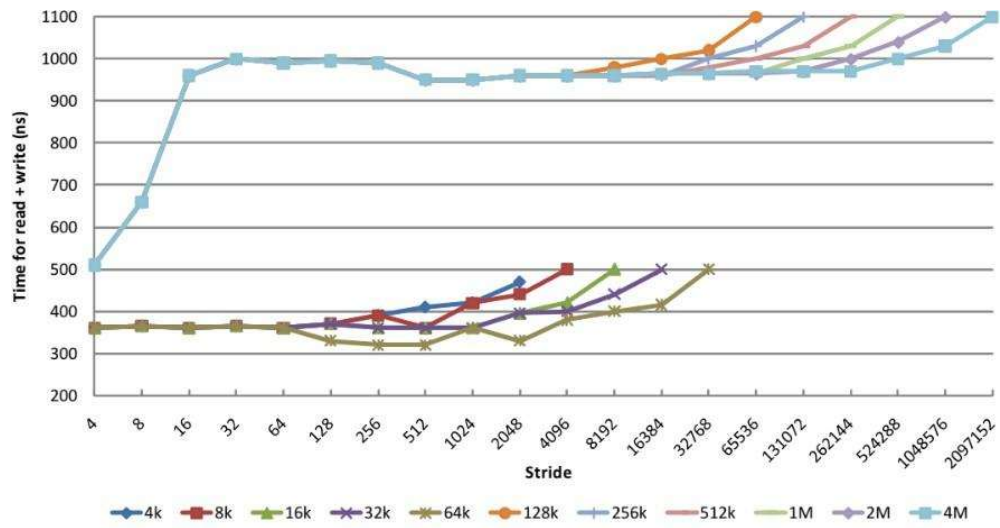


Figure 1: Variation of the cache access time with the adopted *stride* value for different array sizes.

3 Assignment

3.1 Modeling Computer Caches

In the first part of this assignment, the goal is to model the characteristics of the L1 data cache and L2 cache of the targeted computer platform. Next, we provide instructions for performing this analysis.

Use the forms at the end to answer the questions below.

3.1.1 Modeling the L1 Data Cache

The methodology to experimentally model the L1 data cache consists in considering the total amount of data cache misses during the execution of the following code sequence of program `cm1.c`, similar to the program in Section 2. This program can be found in the package `lab1_kit.zip`.

```
for(size_t cache_size = CACHE_MIN; cache_size < CACHE_MAX; cache_size = 2*cache_size) {
    for(size_t stride = 1; stride <= cache_size/2; stride = 2*stride){
        limit = cache_size - stride + 1;
        for(ssize_t repeat = 0; repeat < N_REPETITIONS * stride; repeat++) {
            for(index = 0; index < limit; index += stride) {
                array[index] = array[index] + 1;
            }
        }
    }
}
```

- a) Change to directory `cm1/`, in the package `lab1_kit.zip`, and analyze the code of the program `cm1.c`. Identify its source code with the program described above.

What are the processor events that will be analyzed during its execution? Explain their meaning.

O principal evento monitorizado pelo programa é o **PAPI_L1_DCM**, responsável por contar os *misses* na L1 Data Cache.

Em termos práticos, este contador indica quantas vezes o processador tentou aceder a dados que não estavam presentes na cache L1 durante a execução.

A análise desse evento permite avaliar a utilização da cache L1 em função dos diferentes tamanhos de arrays e *strides*. Além disso, o programa regista também o tempo total de execução, tanto em ciclos de clock como em microsegundos, tornando possível relacionar as falhas na cache com o impacto direto no desempenho.

Em resumo, este evento mede a eficiência da cache L1: um número elevado de *misses* revela uma má utilização da cache, o que conduz a perdas de desempenho, já que os acessos à memória principal são consideravelmente mais lentos do que os acessos à cache.

- b) Compile the program `cm1.c` using the provided `Makefile` and execute `cm1`. **Plot the variation**

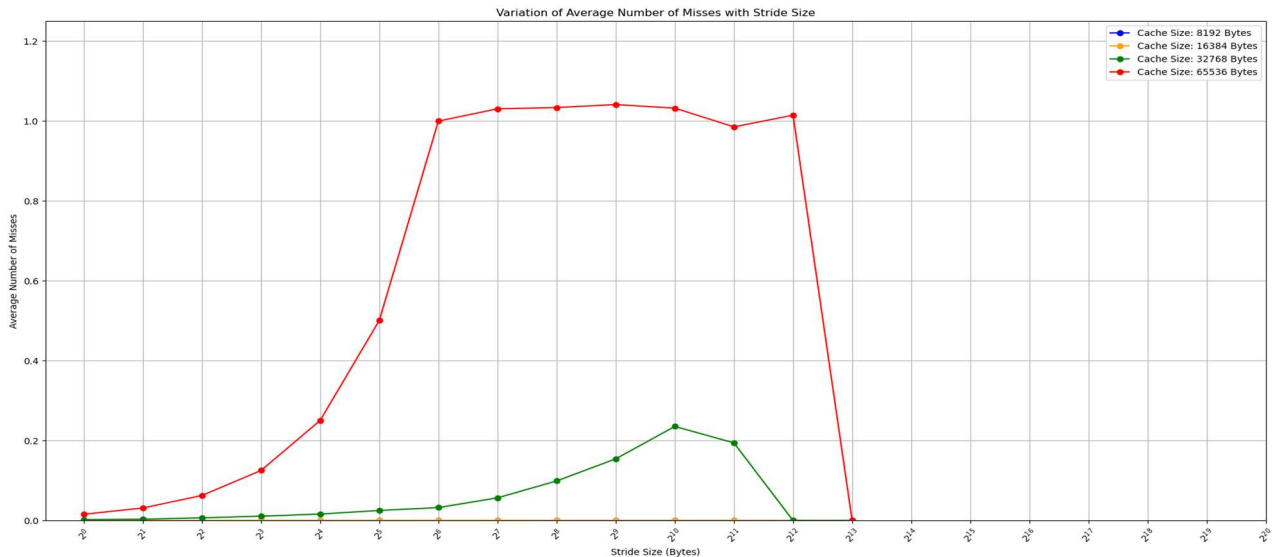
of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L1 data cache (8kB, 16kB, 32kB and 64kB).

NOTE: A fast sketch of these plots can be drawn in your computer by running the following commands:

```
./cml > cml.out  
./cml_proc.sh
```

NOTE 2: You can draw these tables and plots on your computer, print, and attach to the report. You do not have to fill them by hand on the printed report.

NOTE 3: You may need to mark the script as executable before being able to run it.



c) By analyzing the obtained results:

- **Determine the size of the L1 data cache. Justify your answer.**

O comportamento da cache mostra que, para caches de 8 KiB e 32 KiB, as taxas médias de misses permanecem muito baixas, sobretudo quando os strides são pequenos. No entanto, ao observar a cache de 64 KiB, verifica-se um crescimento expressivo no número de misses sempre que os strides ultrapassam 64 bytes. Esse aumento indica que, a partir desse ponto, os dados solicitados já não se encontram disponíveis na cache, forçando acessos à memória principal.

Um exemplo marcante ocorre com stride de 8192 bytes numa cache de 8 KiB: nesse caso, a taxa média de misses é nula, confirmando que todos os dados acessados podem ser armazenados integralmente na cache. A primeira subida relevante nas misses acontece exatamente quando o stride excede a capacidade da cache. Esse comportamento evidencia o limite efetivo da cache L1.

Assim, a análise revela que a cache de dados L1 possui 32 KiB de capacidade. Essa conclusão também é sustentada pelo gráfico, no qual a linha correspondente a 32 KiB é a última a manter taxas de misses médias próximas de zero de forma consistente.

- **Determine the block size adopted in this cache. Justify your answer.**

Para determinar o tamanho do bloco de uma cache, podemos analisar a relação entre o comprimento do stride e os tempos de acesso. À medida que o stride aumenta, observa-se o comportamento do tempo médio de acesso. O ponto em que este valor estabiliza indica que o stride atingiu (ou superou) o tamanho do bloco, fazendo com que acessos sequenciais passem a incidir em blocos distintos da cache, resultando numa taxa de *misses* próxima de 100% para strides seguintes.

No gráfico analisado, verifica-se que para strides inferiores a 64 Bytes a taxa de *misses* continua a crescer. Contudo, a partir do momento em que o stride ultrapassa os 64 Bytes, o tempo médio de acesso estabiliza. Isso significa que o tamanho do bloco de cache corresponde a 64 Bytes, ou seja, 2^6 .

- **Characterize the associativity set size adopted in this cache. Justify your answer.**

Associatividade 8-way

Curva 32kB:

Stride $2^7 = 128\text{B}$ primeiro aumento substancial

Stride $2^8 = 256\text{B}$ pico de conflitos

Total cache = 32kB

Block size = 64B

Nº blocos = $32\text{kB}/64\text{B} = 512$ blocos

Nº sets = $512 / 8 = 64$ sets

Set distance = $64 \text{ sets} * 64 \text{ B} = 4096\text{B}$

Confirmação:

Stride 128:

Distancia entre endereços conflituosos = $128 * 64 = 8192\text{B}$

$8192/4096 = 2$ sets de distancia -> baixo conflito e miss rate baixo

Stride 256:

Distancia entre endereços conflituosos = $256 * 64 = 16384\text{B}$

$16384/4096 = 4$ sets de distancia -> conflitos começam e pico de miss rate

Se fosse 4-way o pico de conflitos apareceria mais cedo, se fosse 16-way a miss rate seria menor no pico

A **8-way associativity** explica perfeitamente o padrão observado onde o miss rate atinge o pico no stride 256.

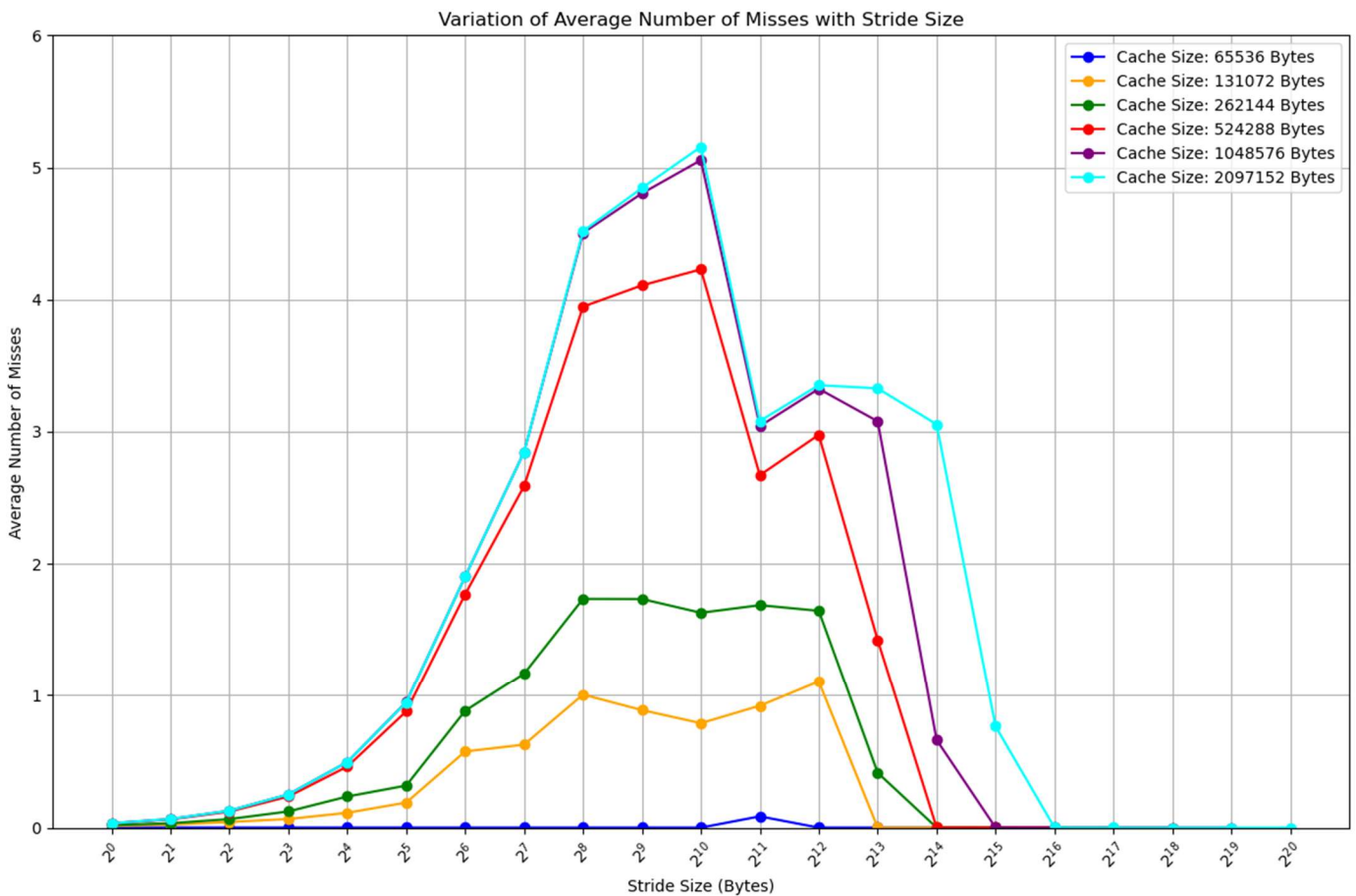
3.1.2 Modeling the L2 Cache

In this part of the assignment, the goal is to experimentally model the characteristics of the L2 cache of the targeted computer platform. To analyze the computer's L2 cache, we will use the same methodology that was introduced in the previous section to model the L1 data cache.

- a) Modify the program `cm1.c` in order to analyze the characteristics of the L2 cache. (Hint: use the event `PAPI_L2_DCM`.) **Describe and justify the changes introduced in this program.**

No nosso programa `cm1.c`, alteramos duas coisas essenciais para analisar as características da cache L2. Começando pelo evento PAPI, mudamos de `PAPI_L1_DCM` para `PAPI_L2_DCM` devido a querermos agora medir a miss rate da cache L2. De seguida alteramos o valor da `CACHE MAX` que foi aumentado para 2 MiB devido a esta cache L2 ser muito maior que a L1, de maneira a ser necessário arrays maiores para ultrapassar o tamanho da cache L2.

- b) Compile the program `cm1.c`, execute `cm1`, and **plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L2 cache.**



c) By analyzing the obtained results:

- **Determine the size of the L2 cache. Justify your answer. 256kB**

Curva 262144 B mostra miss rate relativamente baixo (~1.7 no pico), curvas abaixo mostram miss rates muito baixos, e as curvas acima os miss rates aumentam muito. Ponto de transição entre 256 e 512, logo pode-se concluir que a cache L2 é de 256kB

- **Determine the block size adopted in this cache. Justify your answer.**

Todas as curvas mostram comportamento especial no stride 64 (2^6)

Stride < 64: Miss rates baixos (múltiplos elementos na mesma cache line)

Stride >= 64: Aumento do miss rate (nova cache line por acesso)

Stride 64 representa o tamanho da cache line

- **Characterize the associativity set size adopted in this cache. Justify your answer.**

8-way:

Pico no stride 128-256 B

Miss rate máximo ~1.7

Cálculo:

L2 Cache = 256KB, Block = 64B

Total blocks = $256\text{KB} \div 64\text{B} = 4096$ blocks

Se 8-way:

Sets = $4096 \div 8 = 512$ sets

Set distance = $512 \times 64\text{B} = 32\text{KB}$

3.2 Profiling and Optimizing Data Cache Accesses

Often, programmers wishing to improve their programs' performance focus their attention on how the programs affect the computer's caches. In the following, it will be analyzed how simple code changes can help to improve that performance for a matrix multiplication application.

Consider a simple matrix multiplication application, operating on two square matrices of $N \times N$ 16-bit integer elements, with $N = 1024$. From a mathematical point of view, given two matrices **A** and **B**, with elements a_{ij} and b_{ij} such that $0 \leq i, j < N$, the product matrix **C** is defined as:

$$c_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{kj} = a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{i(N-1)}b_{(N-1)j}$$

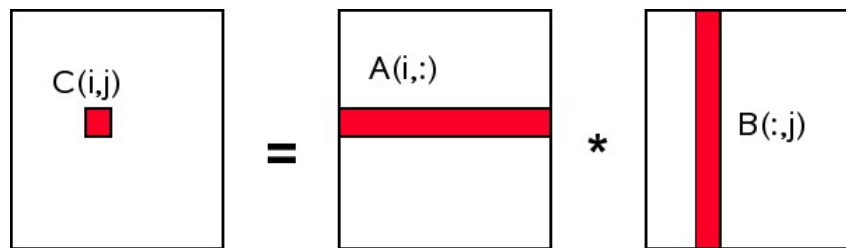


Figure 2: Straightforward matrix multiplication.

3.2.1 Straightforward implementation

A straight-forward C implementation of Eq. 1 can look like this:

```
for (i = 0; i < N; ++i) {
    for (j = 0; j < N; ++j) {
        for (k = 0; k < N; ++k) {
            res[i][j] += mul1[i][k] * mul2[k][j];
        }
    }
}
```

The two input matrices are `mul1` and `mul2`. The result matrix `res` is assumed to be initialized to all zeroes.

The provided program `mm1.c` includes this code sequence and all the necessary initialization steps, as well as the set of statements that are required in order to profile its execution using the PAPI toolbox.

- Change to directory `mm1/` and analyze the code of the program `mm1.c`. Identify its source code with the program described above.

What is the total amount of memory that is required to accommodate each of these matrices?

```
size_of_int16 = 2 # int16_t takes up 2 bytes
array_n = 512 # defined as a constant in mm1.c

matrixsize_in_bytes = size_of_int16 * array_n * array_n

print(
    f"Each matrix takes up {matrixsize_in_bytes} Bytes ({matrixsize_in_bytes >>
10} KiB)."
)
```

Each matrix takes up 524288 Bytes (512 KiB).

- b) Compile the source file `mm1.c` using the provided `Makefile` and execute it. **Fill the table with the obtained data.**
- c) **Evaluate the resulting L1 data cache *Hit-Rate*.**

3.2.2 First Optimization: Matrix transpose before multiplication [2]

By analyzing the obtained results, it can be observed that such a straightforward implementation suffers from a severe penalty in what concerns the amount of L2 cache misses resulting from its access pattern. In fact, while the `mul1` matrix is accessed sequentially, the inner loop advances the row number of `mul2` (see Fig. 2), meaning successive accesses to far away memory positions.

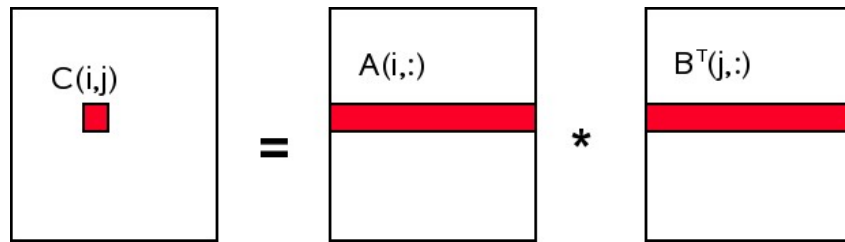


Figure 3: Transposed matrix multiplication.

One possible remedy to attenuate such a problem is based on matrix transposition. In fact, since each matrix element is accessed multiple times, it might be worthwhile to rearrange (“transpose,” in mathematical terms) the second matrix `mul2` before using it (see Fig. 3):

$$c_{ij} = \sum_{k=0}^{N-1} a_{ik} b_{jk}^T = a_{i1} b_{j1}^T + a_{i2} b_{j2}^T + \dots + a_{i(N-1)} b_{j(N-1)}^T$$

After the preliminary transposition step, both matrices may be iterated sequentially. As far as the C code is concerned, it now looks like this:

```
int16_t tmp[N][N];

// transposition
for (i = 0; i < N; ++i) {
    for (j = 0; j < N; ++j) {
        tmp[i][j] = mul2[j][i];
    }
}

// multiplication
for (i = 0; i < N; ++i) {
    for (j = 0; j < N; ++j) {
        for (k = 0; k < N; ++k) {
            res[i][j] += mul1[i][k] * tmp[j][k];
        }
    }
}
```

Variable `tmp` is a temporary array to store the transposed matrix. One direct consequence of this optimization is that it now requires additional access to the data memory. Hopefully, this extra cost can be easily recovered, since the 1024 non-sequential accesses per column are usually much more expensive.

- a) Change to directory `mm2/` and analyze the code of the program `mm2.c`. Identify its source code with the program described above. Compile this program using the provided `Makefile` and execute it. **Fill the table with the obtained data.**
- b) **Evaluate the resulting L1 data cache *Hit-Rate*.**
- c) Change the code in the program `mm2.c` in order to include the matrix transposition in the execution time. Compile this program using the provided `Makefile` and execute it.
Fill the table with the obtained data. Comment on the obtained results when including the matrix transposition in the execution time.
- d) **Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedups.**

3.2.3 Second Optimization: Blocked (tiled) matrix multiply [2]

Despite the good results that may be obtained with the matrix transposition method, in many applications this approach can not be applied, either because the matrix is too large or the available memory is too small. Hence, other alternatives, which do not require the extra copy procedure, should be studied.

The search for an alternative processing scheme should start with a close examination of the involved math and the operations performed by the original implementation. Trivial math knowledge shows that the order of the several additions to obtain each element of the result matrix is irrelevant, as long as each addend appears exactly once. This understanding will lead to solutions which reorder the additions performed in the inner loop of the original code.

According to the original algorithm, the adopted order to access the elements of matrix `mul2` is: (0,0), (1,0), ... , (N-1,0), (0,1), (1,1), Although the elements (0,0) and (0,1) are in the same cache line, by the time the inner loop completes one round, this cache line has long been evicted. For this example, each round of the inner loop requires, for each of the three matrices, 1024 cache lines, which is much more than what is available in most processors' caches.

One possible solution is to simultaneously handle more than one iteration of the middle loop, while executing the inner loop. In this case, several values which are guaranteed to be in cache will be used, thus contributing to a reduction of the L2 cache miss-rate. Hence, to maximize the speedup provided by this technique, it is necessary to adapt the dimension of the sub-matrix under processing to the cache block size, by taking into account the size of each matrix element. As a hypothetical example, considering that a `short` operand occupies 2-Bytes, this means that a 64-Byte cache block will accommodate 32 matrix elements, thus defining the optimal size for the sub-matrix line to be 32 (see Fig. 4).

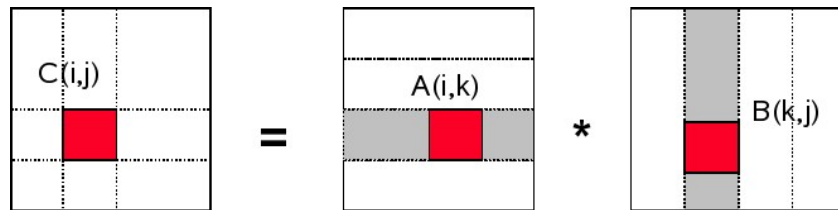


Figure 4: Blocked matrix multiplication.

As far as the C code is concerned, it now looks like this:

```
#define SUB_MATRIX_SIZE (CACHE_LINE_SIZE / sizeof (short))

for (size_t i = 0; i < N; i += SUB_MATRIX_SIZE) {
    for (size_t j = 0; j < N; j += SUB_MATRIX_SIZE) {
        for (size_t k = 0; k < N; k += SUB_MATRIX_SIZE) {
            for (size_t inner_i = 0; inner_i < SUB_MATRIX_SIZE; ++inner_i) {
                for (size_t inner_k = 0; inner_k < SUB_MATRIX_SIZE;
                    ++inner_k) {
                    for (size_t inner_j = 0; inner_j < SUB_MATRIX_SIZE;
                        ++inner_j) {
                        res[i + inner_i][j + inner_j] +=
                            factor1[i + inner_i][k + inner_k] *
                            factor2[k + inner_k][j + inner_j];
                    }
                }
            }
        }
    }
}
```

The most visible change is that the code has six nested loops now. The outer loops iterate with intervals of `SUB_MATRIX_SIZE` (the cache line size `CACHE_LINE_SIZE` divided by `sizeof(short)`). This divides the matrix multiplication into several smaller problems which can be handled with more cache locality. The inner loops iterate over the missing indexes of the outer loops. There are, once again, three loops. The `k2` and `j2` loops are in a different order. This is done because, in the actual computation, only one expression depends on `k2` but two depend on `j2`.

- a) Change to directory `mm3/` and analyze the code of the program `mm3.c`. Identify its source code with the program described above.

Change the program source code in order to comply with the algorithm parameterization (sub-matrix line size) with the block size (CLS) that was determined in Section 3.1.

How many matrix elements can be accommodated in each cache line?

```
m_element_size = 2 # = size_of_int16 = 2 # int16_t takes up 2 bytes
l1_block_size = 64 # bytes
elements_per_cache_line = l1_block_size / m_element_size
print(f"Em cada linha da cache podem estar {int(elements_per_cache_line)}
elementos da matriz.")
```

Em cada linha da cache podem estar 32 elementos da matriz.

- b) Compile this program using the provided `Makefile` and execute it. Fill the table with the obtained data. Evaluate the resulting L1 data cache *Hit-Rate*.
- c) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.
- d) Compare the obtained results with those that were obtained for the matrix transpose implementation by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.

If the obtained speedup is positive, but the difference of the resulting hit-rates is negative, how do you explain the performance improvement?

(Hint: study the hit-rates of the L2 cache for both implementations; You may use the following PAPI events `PAPI_L2_DCH` (or `PAPI_L2_DCM`) and `PAPI_L2_DCA`. Run `papi_avail` to check for available events and understand their meaning.)

References

- [1] Performance Application Programming Interface (PAPI). Webpage. "<http://icl.cs.utk.edu/papi>", December 2008.
- [2] Ulrich Drepper. What every programmer should know about memory. Technical report, Red Hat, Inc., November 2007.
- [3] *PAPI User's Guide*.
- [4] *PAPI Programmer's Reference*.

Second Lab Assignment: System Modeling and Profiling

STUDENTS IDENTIFICATION:

Number:	Name:
57175	João Pedro dos Santos Carvalho
105882	Duarte Mendonça Gouveia
107032	Rodrigo Alexandre Almeida Fonseca dos Santos

2 First steps

Please justify all your answers with values from the experiments.

1. What is the cache capacity of the computer you used (please write the workstation name)?

Array Size	4096	8192	16384	32768	65536	131072
t2-t1(ms)	1.009	2.140	4.013	8.065	13.900	26.813
# accesses a[i]	204800	409600	819200	1638400	3276800	6553600
# mean access time(ns)	4.9267	5.2246	4.8987	4.9225	4.2419	4.0913

O computador usado pertence ao Lab0p8 . Com base na tabela preenchida podemos observar que o tempo médio de acesso se mantém quase constante. No entanto, há um aumento significativo no tempo de acesso entre os arrays de tamanho 32 e 64 e isto indica que a cache já não consegue armazenar todos os dados, o que resulta num maior número de acessos à memória principal. Logo podemos concluir que o tamanho da cache é de 32kB.

Consider the data presented in Figure 1. Answer the following questions (2, 3, 4) about the machine used to generate that data.

2. What is the cache capacity?

A análise da Figura 1 revela a existência de dois comportamentos distintos nos tempos de acesso, consoante o tamanho dos arrays.
No primeiro caso, observa-se um tempo de acesso estável entre 300 ns e 500 ns, característico de arrays de menor dimensão. Este comportamento indica que esses arrays conseguem ser totalmente armazenados na cache, garantindo acessos rápidos e consistentes.
Já no segundo caso, os tempos de acesso sobem para valores aproximadamente entre os 900 ns e os 1000 ns , o que evidencia a ocorrência frequente de cache misses. Isto acontece porque arrays maiores ultrapassam a capacidade da cache, forçando o sistema a recorrer com maior frequência à memória principal.
O ponto de transição entre os dois cenários surge entre arrays de 64 KiB e 128 KiB, onde há um aumento acentuado no tempo de leitura e escrita. Esse salto é um forte indício de que a cache possui 64 KiB de capacidade, dado que esse é o maior tamanho de array que ainda pode ser inteiramente acomodado nela sem penalizações significativas.

3. What is the size of each cache block?

Para identificar o tamanho do bloco de cache, é necessário observar o primeiro valor de stride em que os tempos de acesso do grupo mais lento se tornam constantes.
Esse comportamento ocorre porque, a partir do momento em que o stride é igual ou superior ao tamanho do bloco, cada acesso passa a referir-se a um bloco diferente da cache. Isso provoca uma taxa de miss próxima de 100%, resultando em tempos de acesso uniformes para todos os strides seguintes.
Na análise, essa estabilização acontece a partir de um stride de 16 Bytes, o que indica que o bloco de cache possui exatamente 16 Bytes.

4. What is the L1 cache miss penalty time?

Para calcular a penalização associada a uma falta na cache L1, basta comparar o tempo de acesso quando todos os dados cabem na cache com o tempo de acesso quando há misses e o sistema é obrigado a recorrer à memória principal. A diferença entre estes valores traduz o custo de um cache miss.

Quando o array é menor do que a capacidade da cache, a taxa de miss é praticamente nula e o tempo de acesso ronda os 375 ns. Já para arrays maiores, combinados com strides iguais ou superiores a 16 bytes, a taxa de miss aproxima-se dos 100%, resultando em tempos de acesso em torno de 1000 ns.

Assim, a penalização de uma falta na cache L1 corresponde a $1000 - 375 = 625$ ns (L2 access time - L1 access time) .

3 Assignment

3.1.1 Modeling the L1 Data Cache

- a) What are the processor events that will be analyzed during its execution? Explain their meaning.

O principal evento monitorizado pelo programa é o *PAPI_L1_DCM*, responsável por contar os *misses* na L1 Data Cache.

Em termos práticos, este contador indica quantas vezes o processador tentou aceder a dados que não estavam presentes na cache L1 durante a execução.

A análise desse evento permite avaliar a utilização da cache L1 em função dos diferentes tamanhos de *arrays* e *strides*. Além disso, o programa regista também o tempo total de execução, tanto em ciclos de clock como em microsegundos, tornando possível relacionar as falhas na cache com o impacto direto no desempenho.

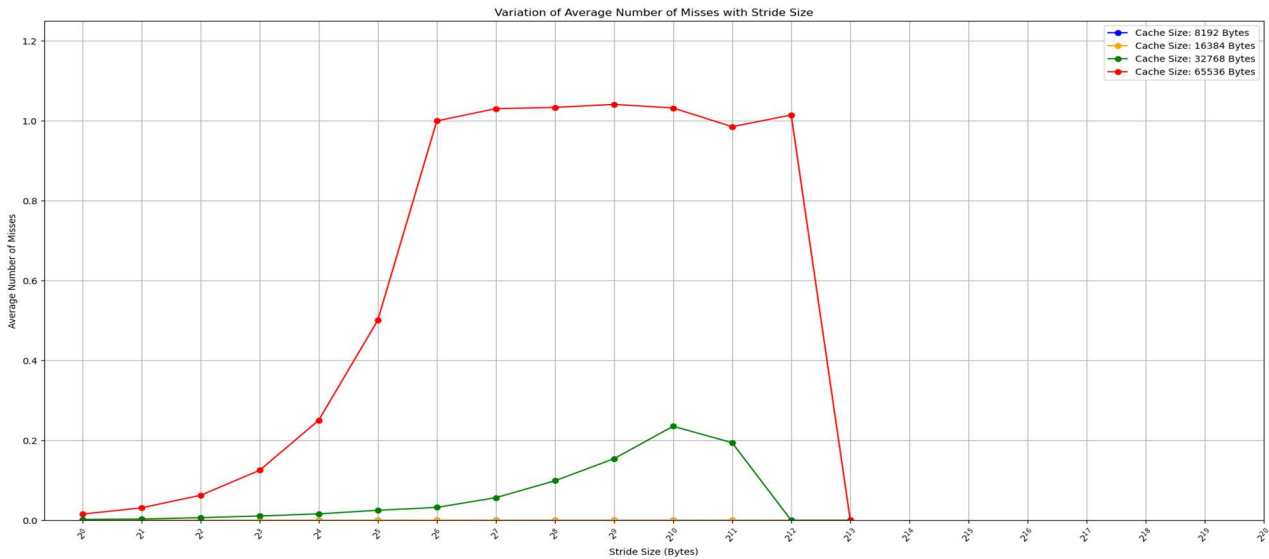
Em resumo, este evento mede a eficiência da cache L1: um número elevado de misses revela uma má utilização da cache, o que conduz a perdas de desempenho, já que os acessos à memória principal são consideravelmente mais lentos do que os acessos à cache.

- b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L1 data cache (8kB, 16kB, 32kB and 64kB).

Note that, you may fill these tables and graphics (as well as the following ones in this report) on your computer and submit the printed version

Array Size	Stride	Avg Misses	Avg Cycl Time
8kBytes	1	0.000271	0.003434
	2	0.000212	0.003444
	4	0.000125	0.003448
	8	0.000102	0.003293
	16	0.000102	0.003214
	32	0.000093	0.002802
	64	0.000078	0.002516
	128	0.000054	0.002253
	256	0.000022	0.002050
	512	0.000021	0.002073
	1024	0.000021	0.002149
	2048	0.000012	0.002361
	4096	0.000010	0.002187
16kBytes	1	0.000314	0.002361
	2	0.000272	0.002366
	4	0.000270	0.002359
	8	0.000236	0.002278
	16	0.000255	0.002353
	32	0.000222	0.002321
	64	0.000275	0.002291
	128	0.000148	0.002287
	256	0.000069	0.002128
	512	0.000046	0.002053
	1024	0.000035	0.002095
	2048	0.000015	0.002317
	4096	0.000006	0.002280
	8192	0.000006	0.002188

Array Size	Stride	Avg Misses	Avg Cycl Time
32kBytes	1	0.002341	0.002332
	2	0.003262	0.002330
	4	0.006639	0.002284
	8	0.010881	0.002240
	16	0.016120	0.002114
	32	0.025183	0.002146
	64	0.032325	0.001989
	128	0.056733	0.002300
	256	0.099014	0.002254
	512	0.154336	0.002098
	1024	0.235484	0.002131
	2048	0.194339	0.002394
	4096	0.000015	0.002308
	8192	0.000004	0.002280
	16384	0.000005	0.002197
64kBytes	1	0.015657	0.002129
	2	0.031272	0.001999
	4	0.062666	0.002259
	8	0.125273	0.002352
	16	0.250514	0.002321
	32	0.500842	0.002218
	64	0.999995	0.001987
	128	1.030769	0.002032
	256	1.033889	0.002121
	512	1.041352	0.002311
	1024	1.032426	0.002623
	2048	0.985518	0.002990
	4096	1.014776	0.005339
	8192	0.000318	0.002310
	16384	0.000002	0.002280
	32768	0.000003	0.002195



c) By analyzing the obtained results:

- Determine the **size** of the L1 data cache. Justify your answer.

O comportamento da *cache* mostra que, para *caches* de 8 KiB e 32 KiB, as taxas médias de *misses* permanecem muito baixas, sobretudo quando os *strides* são pequenos. No entanto, ao observar a *cache* de 64 KiB, verifica-se um crescimento expressivo no número de *misses* sempre que os *strides* ultrapassam 64 bytes. Esse aumento indica que, a partir desse ponto, os dados solicitados já não se encontram disponíveis na cache, forçando acessos à memória principal.

Um exemplo marcante ocorre com *stride* de 8192 bytes numa *cache* de 8 KiB: nesse caso, a taxa média de *misses* é nula, confirmando que todos os dados acedidos podem ser armazenados integralmente na cache. A primeira subida relevante nas *misses* acontece exatamente quando o *stride* excede a capacidade da *cache*. Esse comportamento evidencia o limite efetivo da *cache* L1.

Assim, a análise revela que a *cache* de dados L1 possui 32 KiB de capacidade. Essa conclusão também é sustentada pelo gráfico, no qual a linha correspondente a 32 KiB é a última a manter taxas de *misses* médias próximas de zero de forma consistente.

- Determine the **block size** adopted in this cache. Justify your answer.

Para determinar o tamanho do bloco de uma *cache*, podemos analisar a relação entre o comprimento do *stride* e os tempos de acesso. À medida que o *stride* aumenta, observa-se o comportamento do tempo médio de acesso. O ponto em que este valor estabiliza indica que o *stride* atingiu (ou superou) o tamanho do bloco, fazendo com que acessos sequenciais passem a incidir em blocos distintos da cache, resultando numa taxa de *misses* próxima de 100% para *strides* seguintes.

No gráfico analisado, verifica-se que para *strides* inferiores a 64 Bytes a taxa de *misses* continua a crescer. Contudo, a partir do momento em que o *stride* ultrapassa os 64 Bytes, o tempo médio de acesso estabiliza. Isso significa que o tamanho do bloco de *cache* corresponde a 64 Bytes, ou seja, 2^6 .

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

Para determinar a associatividade da *cache*, devemos analisar como diferentes *strides* afetam a *miss rate*, concentrando-nos especificamente na *cache* de 64kB representada pela curva vermelha no gráfico.

A chave está em compreender como os endereços de memória são mapeados para conjuntos de *cache* (*cache sets*). Numa cache de 64kB com blocos de 64B temos um total de 128 *sets* com uma distância entre conjuntos de 8kB.

Isto significa que endereços separados por múltiplos de 8kB irão mapear para o mesmo set de cache, criando potenciais conflitos quando há mais de 8 elementos a competir pelo mesmo *set*. Observando o gráfico verifica-se que, para *strides* pequenos (2^0 a 2^{11}) a *miss rate* permanece baixa (os elementos acedem a conjuntos diferentes, evitando conflitos sistemáticos), para o *stride* 2^{12} ainda se observa um *miss rate* baixo, pois é metade da distância crítica de 8kB.

Para o *stride* 2^{13} aqui surge um aumento significativo na *miss rate*. Este é o ponto crítico onde os elementos começam a mapear sistematicamente para os conjuntos de *cache*, forçando conflitos entre mais de 8 elementos por conjunto.

Para *strides* maiores a *miss rate* mantém-se elevada ou aumenta, confirmando que ultrapassámos o limite de associatividade.

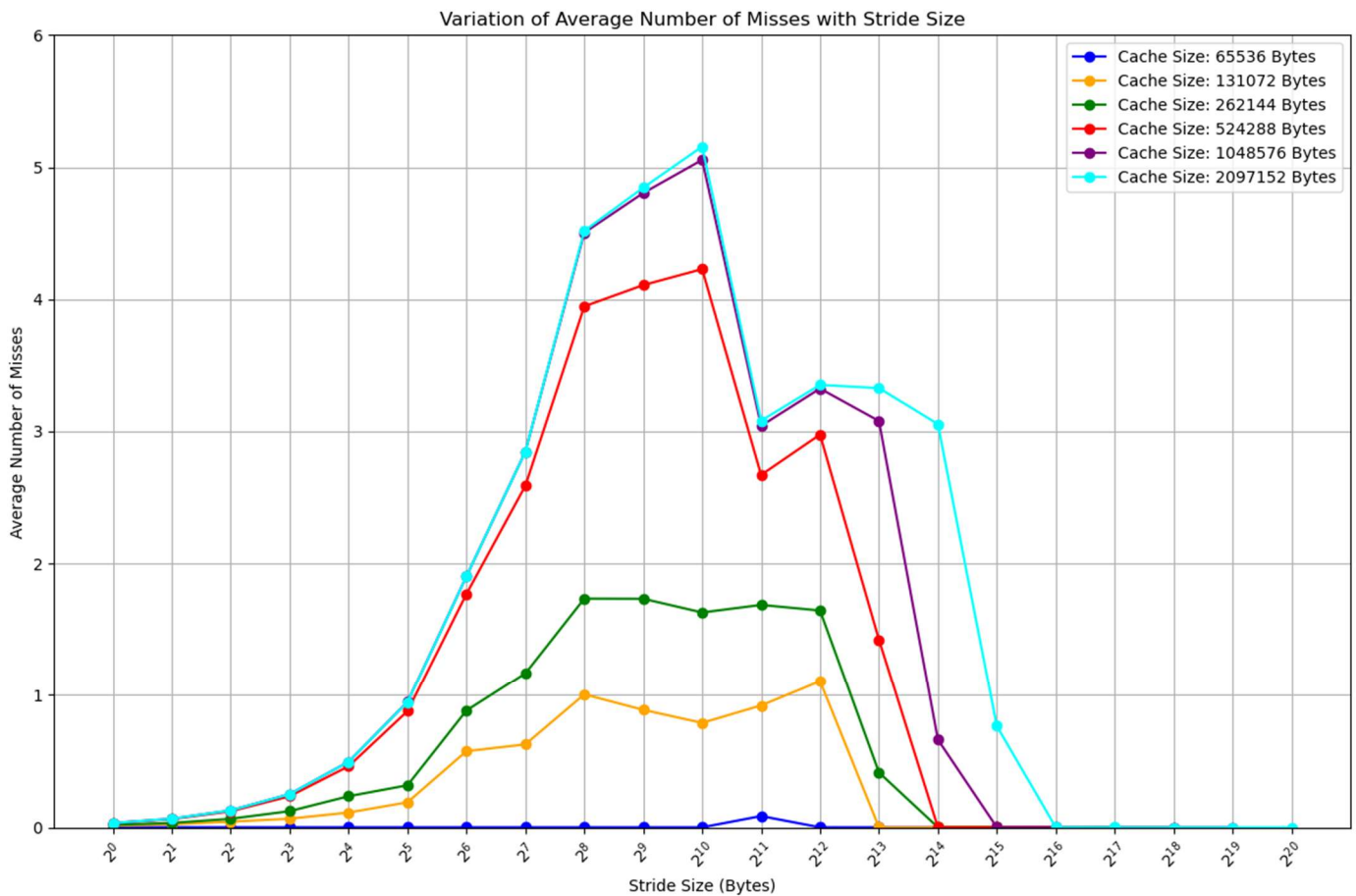
Podemos concluir, então, que como há o aumento de *miss rate* para o *stride* 2^{13} , a *cache* possui uma associatividade de 8 vias, se fosse de 4 vias, a *miss rate* aumentaria mais cedo (nomeadamente no *stride* 2^{12}) e se fosse de 16 vias o aumento ocorreria no *stride* 2^{14} .

3.1.2 Modeling the L2 Cache

- a) Describe and justify the changes introduced in this program.

No nosso programa `cm1.c`, alteramos duas coisas essenciais para analisar as características da *cache* L2. Começando pelo evento PAPI, mudamos de `PAPI_L1_DCM` para `PAPI_L2_DCM` devido a querermos agora medir a miss rate da *cache* L2. De seguida alteramos o valor da `CACHE_MAX` que foi aumentado para 2 MiB devido a esta *cache* L2 ser muito maior que a L1, de maneira a ser necessário *arrays* maiores para ultrapassar o tamanho da *cache* L2.

- b) Plot the variation of the average number of misses (*Avg Misses*) with the `stride` size, for each considered dimension of the L2 cache.



c) By analyzing the obtained results:

- Determine the **size** of the L2 cache. Justify your answer.

O comportamento observado nas curvas confirma o impacto do tamanho da *cache* L2 na taxa de *misses*. A curva correspondente a 262 144 B (256 KiB) apresenta uma taxa de *misses* relativamente baixa, atingindo um pico em torno de 1,7%, mas mantendo-se sempre abaixo das curvas superiores. Já as curvas que representam tamanhos inferiores a esse valor exibem taxas de *misses* ainda menores, praticamente desprezíveis, enquanto as curvas acima de 256 KiB revelam um aumento acentuado no número de *misses*.

Esse padrão mostra um ponto de transição claro entre 256 KiB e 512 KiB, indicando que a *cache* consegue comportar eficazmente os dados até o limite de 256 KiB, mas, a partir desse ponto, os acessos passam a exigir com mais frequência consultas à memória principal.

Dessa forma, a análise permite concluir que a *cache* L2 possui 256 KiB de capacidade, conclusão que também é corroborada pelo comportamento das curvas no gráfico, onde a linha correspondente a 262 144 B é a última a manter um nível de *misses* relativamente baixo de forma consistente.

- Determine the **block size** adopted in this cache. Justify your answer.

Para identificar o tamanho da *cache line*, analisamos o comportamento das curvas em função do *stride*. Observa-se que todas as curvas apresentam um ponto característico no *stride* de 64 bytes (2^6).

- Para *strides* inferiores a 64 bytes, as taxas de *miss* mantêm-se muito baixas, pois vários elementos consecutivos ainda são carregados dentro da mesma linha de *cache*.
- Quando o *stride* atinge ou ultrapassa 64 bytes, nota-se um aumento expressivo no *miss rate*, já que cada acesso passa a incidir em uma nova *cache line*, eliminando o reaproveitamento dos dados previamente carregados.

Assim, o comportamento das curvas confirma que o tamanho da *cache line* é de 64 bytes (2^6).

- Characterize the **associativity set size** adopted in this cache. Justify your answer.

Para determinar a associatividade da cache, devemos analisar como diferentes *strides* afetam a *miss rate*, concentrando-nos especificamente na cache de 256kB representada pela curva verde no gráfico.

A chave está em compreender como os endereços de memória são mapeados para conjuntos de cache (*cache sets*). Numa *cache* de 256kB com blocos de 64B temos um total de 512 *sets* com uma distância entre conjuntos de 32kB.

Isto significa que endereços separados por múltiplos de 32kB irão mapear para o mesmo *set* de *cache*, criando potenciais conflitos quando há mais de 8 elementos a competir pelo mesmo *set*.

Observando o gráfico verifica-se que, para *strides* pequenos (2^0 a 2^{14}) a *miss rate* permanece baixa (os elementos acedem a conjuntos diferentes, evitando conflitos sistemáticos), para o *stride* 2^{14} ainda se observa um *miss rate* baixo, pois é metade da distância crítica de 32kB.

Para o *stride* 2^{15} aqui surge um aumento significativo na *miss rate*. Este é o ponto crítico onde os elementos começam a mapear sistematicamente para os conjuntos de *cache*, forçando conflitos entre mais de 8 elementos por conjunto.

Para *strides* maiores a *miss rate* mantém-se elevada ou aumenta, confirmando que ultrapassámos o limite de associatividade. Podemos concluir, então, que como há o aumento de *miss rate* para o *stride* 2^{15} , a *cache* possui uma associatividade de 8 vias, se fosse de 4 vias, a *miss rate* aumentaria mais cedo (nomeadamente no *stride* 2^{14}) e se fosse de 16 vias o aumento ocorreria no *stride* 2^{16} .

3.2 Profiling and Optimizing Data Cache Accesses

3.2.1 Straightforward implementation

- a) What is the total amount of memory that is required to accommodate each of these matrices?

Cada matriz usa 524288 Bytes (512KiB).

- b) Fill the following table with the obtained data.

Total number of L1 data cache misses	135.296337×10^6
Total number of load / store instructions completed	$(402.653839 + 134.218058)$ 536.871897×10^6
Total number of clock cycles	724.541884×10^6
Elapsed time	0.212601 seconds

- c) Evaluate the resulting L1 data cache *Hit-Rate*:

Hit-Rate = (Load/Store Instructions - L1_DCM) / Load/Store Instructions
Hit-Rate = $401.575560 / 536.871897$
Hit-Rate = $0.7481 = 74.81\%$

3.2.2 First Optimization: Matrix transpose before multiplication [2]

- a) Fill the following table with the obtained data.

Total number of L1 data cache misses	4.217232×10^6
Total number of load / store instructions completed	$(402.653839 + 134.218058)$ 536.871897×10^6
Total number of clock cycles	646.579018×10^6
Elapsed time	0.189725 seconds

- b) Evaluate the resulting L1 data cache *Hit-Rate*:

Hit-Rate = (Load/Store Instructions - L1_DCM) / Load/Store Instructions
Hit-Rate = $532.654665 / 536.871897$
Hit-Rate = 0.9921 = 99.21%

- c) Fill the following table with the obtained data.

Total number of L1 data cache misses	4.484082×10^6
Total number of load / store instructions completed	$(402.915984 + 134.480203)$ 537.396187×10^6
Total number of clock cycles	636.017796×10^6
Elapsed time	0.186625 seconds

Comment on the obtained results when including the matrix transposition in the execution time:

Comparando os dois conjuntos de resultados, constata-se que o tempo de execução permanece praticamente inalterado. Este facto evidencia que a transposição matricial é consideravelmente mais eficiente que a operação de multiplicação, explicando por que razão a inclusão da transposição no cálculo do tempo de execução não afecta substancialmente a performance global do algoritmo.

- d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedups.

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm2}} - \text{HitRate}_{\text{mm1}}: 0.9921 - 0.7481 = 0.2440 = 24.40 \%$
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm1}} / \#\text{Clocks}_{\text{mm2}}: 724.541884 / 646.579018 = 1.12057748$
$\text{Speedup}(\text{Time}) = \text{Time}_{\text{mm1}} / \text{Time}_{\text{mm2}}: 0.212601 / 0.189725 = 1.12057452$
Comment: Obtivemos um speedup de 1.120 em comparação à implementação original, acompanhado de um aumento significativo da <i>hit rate</i> de 24.4%. Esta melhoria deve-se principalmente à redução de L1 <i>cache misses</i>. Na implementação otimizada (mm2), 99.21% dos acessos encontram os dados na <i>cache</i> L1, comparado com apenas 74.81% na versão original. Isto significa que muito menos acessos precisam de ir à <i>cache</i> L2, reduzindo a penalização média por acesso à memória. O <i>speedup</i> relativamente modesto (12%) face à melhoria substancial no <i>hit rate</i> (24.4%) explica-se pelo facto de que o tempo total inclui tanto operações de memória como computação, e os L1 <i>misses</i> são parcialmente compensados pelo acesso rápido à <i>cache</i> L2.

3.2.3 Second Optimization: Blocked (tiled) matrix multiply [2]

- a) How many matrix elements can be accommodated in each cache line?

Em cada linha da *cache* podem estar 32 elementos da matriz.

- b) Fill the following table with the obtained data.

Total number of L1 data cache misses	3.754597×10^6
Total number of load / store instructions completed	$(403.055268 + 134.492511)$ 537.547779×10^6
Total number of clock cycles	281.671526×10^6
Elapsed time	0.082650 seconds

- c) Evaluate the resulting L1 data cache *Hit-Rate*:

Hit-Rate = (Load/Store Instructions - L1_DCM) / Load/Store Instructions
Hit-Rate = $533.793182 / 537.547779$
Hit-Rate = 0.9930 = 99.30%

- d) Compare the obtained results with those that were obtained for the straightforward implementation, by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup.

$\Delta\text{HitRate} = \text{HitRate}_{\text{mm3}} - \text{HitRate}_{\text{mm1}} : 0.9930 - 0.7481 = 0.2449 = 24.49 \%$
$\text{Speedup}(\#\text{Clocks}) = \#\text{Clocks}_{\text{mm1}} / \#\text{Clocks}_{\text{mm3}} : 724.541884 / 281.671526 = 2.57229367$
Comment: Um <i>speedup</i> de 2.57 foi obtido através da implementação de <i>cache blocking</i>, uma técnica que divide a matriz em sub-matrizes que cabem perfeitamente na <i>cache</i> L1. Esta abordagem assegura que todos os dados necessários para os cálculos de cada bloco permanecem na L1 durante essas operações, maximizando a localidade espacial e temporal. Como resultado, conseguiu-se um aumento da <i>hit rate</i> de 24.49%. Esta otimização deve-se principalmente ao facto de que, ao manter os dados de trabalho dentro da capacidade da L1(32kB), praticamente todos os acessos (99.30%) encontram os dados necessários no nível mais rápido da hierarquia de memória, evitando as penalizações associadas aos acessos à L2 ou níveis inferiores. O <i>speedup</i> substancial (157% de melhoria) face à melhoria similar no <i>hit rate</i> comparativamente ao mm2 explica-se pela organização algorítmica superior do <i>cache blocking</i>, que não só reduz <i>cache misses</i> mas também melhora a eficiência computacional através de melhor reutilização de dados. A principal desvantagem desta otimização é o aumento na complexidade do código, que se torna menos intuitivo e mais difícil de manter do que a implementação original.

- e) Compare the obtained results with those that were obtained for the matrix transpose implementation by calculating the difference of the resulting hit-rates ($\Delta\text{HitRate}$) and the obtained speedup. If the obtained speedup is positive, but the difference of the resulting hit-rates is negative, how do you explain the performance improvement? (Hint: study the hit-rates of the L2 cache for both implementations;)

$$\Delta \text{HitRate} = \text{HitRate}_{\text{mm3}} - \text{HitRate}_{\text{mm2}}: 0.9930 - 0.9921 = 0.0009 = 0.09 \%$$

$$\text{Speedup}(\# \text{Clocks}) = \# \text{Clocks}_{\text{mm2}} / \# \text{Clocks}_{\text{mm3}}: 646.579018 / 281.671526 = 2.29550721$$

Comment:

Um *speedup* de 2.30 foi obtido ao comparar mm3 com mm2, acompanhado de uma variação mínima na *hit rate* na L1 (-0.09%) mas com uma melhoria substancial na *hit rate* na L2 de 55.79%. Esta situação demonstra que o mm3 implementa uma estratégia de gestão de memória hierárquica superior.

A ligeira diminuição na *hit rate* da L1 (99.30% vs 99.21%) é mais do que compensada pela melhoria dramática na L2, onde mm3 apresenta uma *hit rate* 55.79% superior à mm2. Esta otimização significa que quando ocorrem falhas na L1 em mm3, os dados são encontrados na L2 com muito maior frequência, evitando acessos aos níveis mais lentos da hierarquia de memória (L3 ou RAM).

O *speedup* significativo (130% de melhoria) explica-se principalmente pela redução das penalizações associadas às falhas na L1. Embora mm3 tenha ligeiramente mais falhas na L1 que mm2, essas falhas são resolvidas muito mais rapidamente na L2, resultando numa penalização média por falha substancialmente menor.

Esta abordagem representa uma otimização equilibrada que não apenas considera a L1 isoladamente, mas otimiza toda a hierarquia de memória.

3.2.3 Comparing results against the CPU specifications

Now that you have characterized the cache on your lab computer, you are going to compare it against the manufacturer's specification. For this you can check the device's datasheet, or make use of the command `lscpu`. Comment on the results.

Após a caracterização experimental da hierarquia de *cache* do sistema laboratorial, procedeu-se à validação dos resultados obtidos comparando-os com as especificações técnicas fornecidas pelo fabricante do processador Intel i5-7500.

A análise comparativa revela uma concordância notável entre os valores experimentais e as especificações oficiais. A *cache* L1 de dados apresentou uma capacidade de 32kB, valor que corresponde exatamente às especificações do fabricante. Da mesma forma, a *cache* L2 demonstrou uma capacidade de 256kB, confirmando as características técnicas documentadas. O tamanho da linha de *cache* de 64B e a associatividade de 8 vias identificados experimentalmente são igualmente consistentes com a arquitetura padrão deste processador. Esta correspondência entre os resultados experimentais e as especificações do fabricante valida tanto a metodologia empregue nos testes de caracterização como a precisão das técnicas de análise utilizadas.

A PAPI - Performance Application Programming Interface

The PAPI project [1] specifies a standard Application Programming Interface (API) for accessing hardware performance counters available in most modern microprocessors. These counters exist as a small set of registers that count *Events*, defined as occurrences of specific signals related to the processor's function (such as cache misses and floating point operations), while the program executes on the processor. Monitoring these events may have a variety of uses in the performance analysis and tuning of an application, since it facilitates the correlation between the source/object code structure and the efficiency of the actual mapping of such code to the underlying architecture. Besides performance analysis, and hand tuning, this information may also be used in compiler optimization, debugging, benchmarking, monitoring and performance modeling.

PAPI has been implemented on a number of different platforms, including: Alpha; MIPS R10K and R12K; AMD Athlon and Opteron; Intel Pentium II, Pentium III, Pentium M, Pentium IV, Itanium 1 and Itanium 2; IBM Power 3, 4 and 5; Cell; Sun UltraSparc I, II and II, etc.

Although each processor has a number of events that are native to that specific architecture, PAPI provides a software abstraction of these architecture-dependent *Native Events* into a collection of *Preset Events*, also known as *predefined events*, that define a common set of events deemed relevant and useful for application performance tuning. These events are typically found in many CPUs that provide performance counters. They give access to the memory hierarchy, cache coherence protocol events, cycle and instruction counts, functional unit, and pipeline status. Hence, preset events may be regarded as mappings from symbolic names (PAPI preset name) to machine specific definitions (native countable events) for a particular hardware resource. For example, Total Cycles (in user mode) is mapped into PAPI_TOT_CYC. Some presets are derived from the underlying hardware metrics. For example, Total L1 Cache Misses (PAPI_L1_TCM) is the sum of L1 Data Misses and L1 Instruction Misses on a given platform. The list of preset and native events that are available on a specific platform can be obtained by running the commands `papi_avail` and `papi_native_avail`, both provided by the papi source distribution.

Besides the standard set of events for application performance tuning, the PAPI specification also includes both a high-level and a low-level sets of routines for accessing the counters. The high level interface consists of eight functions that make it easy to get started with PAPI, by simply providing the ability to start, stop, and read sets of events. This interface is intended for the acquisition of simple but accurate measurement by application engineers [3, 4]:

- `PAPI_num_counters` – get the number of hardware counters available on the system;
- `PAPI_flops` – simplified call to get Mflops/s (floating point operation rate), real and processor time;
- `PAPI_ipc` – gets instructions per cycle, real and processor time;
- `PAPI_accum_counters` – add current counts to array and reset counters;
- `PAPI_read_counters` – copy current counts to array and reset counters;
- `PAPI_start_counters` – start counting hardware events;
- `PAPI_stop_counters` – stop counters and return current counts.

The following is a simple code example of using the high-level API [3, 4]:

```

#include <papi.h>

#define NUM_FLOPS 10000
#define NUM_EVENTS 1

int main(){
    int Events[NUM_EVENTS] = {PAPI_TOT_INS};
    long_long values[NUM_EVENTS];

    /* Start counting events */
    if (PAPI_start_counters(Events, NUM_EVENTS) != PAPI_OK)
        handle_error(1);

    do_some_work();

    /* Read the counters */
    if (PAPI_read_counters(values, NUM_EVENTS) != PAPI_OK)
        handle_error(1);

    printf("After reading the counters: %lld\n", values[0]);

    do_some_work();

    /* Add the counters */
    if (PAPI_accum_counters(values, NUM_EVENTS) != PAPI_OK)
        handle_error(1);

    printf("After adding the counters: %lld\n", values[0]);

    do_some_work();

    /* Stop counting events */
    if (PAPI_stop_counters(values, NUM_EVENTS) != PAPI_OK)
        handle_error(1);

    printf("After stopping the counters: %lld\n", values[0]);
}

```

Possible output:

```

After reading the counters: 441027
After adding the counters: 891959
After stopping the counters:
443994

```

The fully programmable low-level interface provides more sophisticated options for controlling the counters, such as setting thresholds for interrupt on overflow, as well as access to all native counting modes and events. Such an interface is intended for third-party tool writers or users with more sophisticated needs.

The PAPI specification also provides access to the most accurate timers available on the platform in use. These timers can be used to obtain both real and virtual time on each supported platform: the real time clock runs all the time (e.g., a wall clock), while the virtual time clock runs only when the processor is running in user mode.

In the following code example, `PAPI_get_real_cyc()` and `PAPI_get_real_usec()` are used to obtain the real time it takes to create an event set in clock cycles and in microseconds, respectively [3, 4]:

```

#include <papi.h>

int main(){
    long long start_cycles, end_cycles, start_usec, end_usec;
    int EventSet = PAPI_NULL;

    if (PAPI_library_init(PAPI_VER_CURRENT) != PAPI_VER_CURRENT)
        exit(1);

    /*Create an EventSet */
    if (PAPI_create_eventset(&EventSet) != PAPI_OK)
        exit(1);

    /* Gets the starting time in clock cycles */
    start_cycles = PAPI_get_real_cyc();

    /* Gets the starting time in microseconds */
    start_usec = PAPI_get_real_usec();

    do_some_work();

    /* Gets the ending time in clock cycles */
    end_cycles = PAPI_get_real_cyc();

    /* Gets the ending time in microseconds */
    end_usec = PAPI_get_real_usec();

    printf("Wall clock cycles: %lld\n", end_cycles - start_cycles);
    printf("Wall clock time in microseconds: %lld\n", end_usec - start_usec);
}

```

Possible output:

```

Wall clock cycles: 100173
Wall clock time in microseconds: 136

```